

How to Code with Claude

Claude Code で始める研究プログラミングの基礎

セットアップ・基本操作・CLAUDE.md・Skills・Subagents — ハンズオン 4 回・60 分

生体情報システム研究室 セミナー

山口大学 小林

この60分で到達したい状態

- Claude Code を自分のPCで起動し、Python の解析コードを「指示して書かせる」ことができる
- CLAUDE.md と Skills の仕組みを理解し、研究室で共有できる形に整えられる
- 安全に使うためのルール（権限・レビュー・データの扱い）を説明できる

本日のハンズオン（4回）

1

ハンズオン ① 起動と /init

CLAUDE.md を自動生成して読む

2

ハンズオン ② 解析スクリプトに機能追加

欠損値処理を追加し pytest を通す

3

ハンズオン ③ Skill を作る

ラボ規約で図を描く /plot-style

4

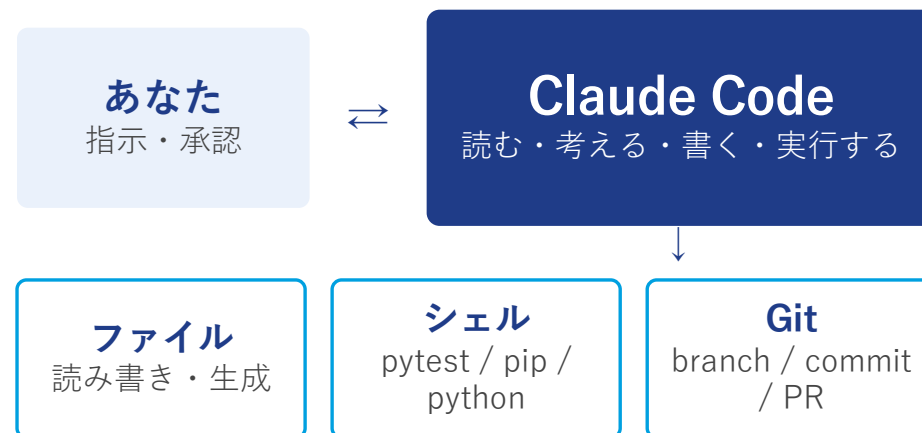
ハンズオン ④ Subagent に分担させる

レビュー担当・テスト担当の subagent を定義

前提：Python の基本とターミナル操作ができること／Claude Code（有料プラン）または Google Antigravity（無料枠）のどちらかが使えること（未了の方は \$1 で一緒に進めます）

ターミナルで動く自律型の AI コーディング エージェント

- 目的を日本語で伝えとコードベースを読み、計画を立て、複数ファイルを編集し、テストを実行
- 1行ずつの補完ではなく、タスク単位で任せる
- 変更はすべて差分として提示し、ユーザが承認してから反映



コード補完ツール (Copilot 等)

- カーソル位置の次の数行を予測
- 設計・実行・検証は人間が行う
- エディタの中だけで完結

Claude Code

- タスク全体（調査→実装→テスト）を委任
- コマンドを実行して結果を見て自己修正
- CLAUDE.md / Skills でAI使用のノウハウを共有

提供形態：ターミナル CLI（本日の主役・macOS / WSL で動作確認済み）／デスクトップアプリ／VS Code・JetBrains 拡張。開発元は Anthropic。

- ・ 研究のコーディングは「調べる・考える」時間が重要。
- ・ 「書く、チェックする」時間を丸ごと任せられる

解析スクリプトの作成

「この CSV を読み、群ごとに平均±SDを出して図にして」
で pandas + matplotlib のコードが出てくる

既存コードの理解・改修

先輩が残した数千行のコードを読ませ、構造を説明させ、
安全に機能追加する

図・表の整形

論文投稿規定（フォント・サイズ・カラー）に合わせて全
図を一括修正

テスト・再現性の確保

pytest の追加、環境ファイル（pyproject / requirements）
の整備、README の生成

定型作業の自動化

データ整形・レポート生成・コミットメッセージなど、毎
回同じ手順を Skill 化

レビューと学習

書いたコードの問題点を説明させる。

コードを書く速度が上がるだけでなく「面倒で先送りしていた整備作業」, 「とりあえず試してみたいパイロット研究」が進む

§1 セットアップ：Claude Code と Antigravity

5

どちらもネイティブインストーラ 1 行 + ブラウザ認証で完了。手元の環境に合う方を選ぶ

		Claude Code 有料プラン（Pro 以上）または API キー	Antigravity CLI (agy) Google アカウントのみ（無料枠・カード不要）
1	プロキシ	<code>export HTTPS_PROXY=http://HOST:PORT # ~/.zshrc (macOS) / ~/.bashrc (WSL) に書く</code>	
2	インストール macOS / WSL	<code>curl --proxy http://HOST:PORT -fsSL ¥ https://claude.ai/install.sh bash</code>	<code>curl --proxy http://HOST:PORT -fsSL ¥ https://antigravity.google/cli/install.sh bash</code>
3	確認・認証 初回はブラウザ	<code>claude --version claude doctor # 診断</code>	<code>agy --version agy # /model で Claude Sonnet 4.6</code>
4	取得	<code>git clone https://github.com/<GitHub アカウント名>/How_to_Code_with_CLAUDE_kobayashi.git sample-analysis cd sample-analysis # claude / agy の起動はハンズオン①（P8）で行う。ここでは取得まで △リポジトリ内のPDFは資料なので、コピー後削除してください。Initの邪魔になります。</code>	

Claude Code の要件

Pro 以上のプラン、または Console の API キー

Antigravity の注意

週次クォータ制で上限は変動。通常は Sonnet を使う

HOST:PORT は当日板書。Antigravity は AGENTS.md に「@CLAUDE.md を読む」と書けばルールを共用できる。詳しい対応表は次スライド。

Claude Code と Antigravity の対応表（両方の参加者向け）

6

道具は違っても「ルールをファイルに書く／手順を Skill にする／役割を分ける」の考え方は同じ

項目	Claude Code（有料プラン）	Antigravity（無料枠あり）
動作形態	ターミナル常駐の CLI（VS Code 拡張もあり）	デスクトップアプリ／CLI。Google アカウントでサインイン
使えるモデル	Claude（Sonnet / Opus）	Gemini・Claude Sonnet/Opus・GPT-OSS を選択可
プロジェクトのルール	CLAUDE.md（/init で生成）	AGENTS.md または .agents/rules/（@CLAUDE.md で参照も可）
個人のルール	~/.claude/CLAUDE.md	~/.gemini/GEMINI.md
Skills	.claude/skills/<名前>/SKILL.md	.agents/skills/<名前>/SKILL.md（同じ SKILL.md 形式）
Subagents	.claude/agents/<名前>.md	対応あり（定義形式は異なる。本日はデモ観察で OK）
差分の承認	ターミナルで y/n・差分表示	アプリ内の差分ビューで承認
ハンズオン①～④	すべて実施	①②③は実施可、④はデモ観察＋概念理解

注意：Antigravity は 2026 年 8 月に 2.0 へ移行中（旧 IDE も当面利用可）。無料枠の上限は変わりうるので、当日は Sonnet を使い Opus は必要なときだけ。

sample-analysis の中で、3 通りのどれかで環境構築（依存は pyproject.toml）

venv	標準機能で完結（推奨） 追加導入不要。Python 3.9 以上	<pre>python3 -m venv .venv source .venv/bin/activate pip install --proxy http://HOST:PORT -e . pytest ruff # (stats.py の実行時には) python stats.py</pre>
uv	速い・Python 本体も管理 uv sync で .venv を自動作成	<pre>curl --proxy http://HOST:PORT -Lsf astral.sh/uv/install.sh sh uv sync # (stats.py の実行時には) uv run python stats.py</pre>
conda	バイナリ依存が多い人向け miniforge 推奨。env 内で pip	<pre>conda create -n sample-analysis python=3.11 -y conda activate sample-analysis pip install --proxy http://HOST:PORT -e . pytest ruff # (stats.py の実行時には) python stats.py</pre>

環境は CLAUDE.md に書いておく

「Python は .venv/bin/python (conda なら env sample-analysis) を使う」 「pip / uv add / conda install は ask」

HOST:PORT は当日板書。プロキシは export HTTPS_PROXY=http://HOST:PORT を ~/.zshrc (macOS) / ~/.bashrc (WSL) に書いておくと毎回の指定が不要。

ハンズオン ①：起動して /init する

8

ゴール：自分の Python プロジェクトで Claude Code を起動し、CLAUDE.md を自動生成して中身を読む

1	配布した sample-analysis リポジトリに移動して起動	<pre>cd sample-analysis claude</pre>
2	プロジェクトを解析させて CLAUDE.md を生成	<pre>/init</pre>
3	生成された内容を確認（別ターミナルでも OK）	<pre>cat CLAUDE.md</pre>
4	Claude に質問してみる	このプロジェクトの構成と、テストの実行方法を教えて

観察ポイント

- 起動時に「このフォルダを信頼しますか？」と聞かれる → 信頼できるリポジトリのみ Yes
- /init は README・pyproject・ディレクトリ構造を読んで CLAUDE.md を書く
- 生成物は「たたき台」。研究室固有のルールは §3 で手で追記する
- Esc で中断、Ctrl+C ×2 で終了

sample-analysis：CSV 読み込み（load_data.py）＋ 集計（stats.py）＋ pytest 1本 ＋ pyproject.toml の最小構成。P5 で git clone 済みのフォルダを使います。

Antigravity の場合：フォルダを開いて「このプロジェクトの構成を調べて AGENTS.md を作って」と頼む（/init の代わり）。生成物を読む作業は同じ。

§2 基本操作：1回のセッションはこう流れる

9

「指示 → 調査 → 計画 → 差分提示 → 承認 → 実行・検証」のループ。承認の関門はあなたが握っている



権限確認 (Permission)

- ファイル編集・コマンド実行の前に必ず確認が出る
- 選択肢：Yes / Yes, and don't ask again for this session / No + 指示
- 設定ファイル `.claude/settings.json` で `allow / ask / deny` を定義 → §4 で詳述
- ヒント：`pytest`・`ruff` は `allow`、`rm`・`git push --force` は `deny` にしておく

プランモード (Shift + Tab で切替)

- 「読む・考える」だけで、ファイルを一切変更しないモード
- 複雑なタスクは まずプランモードで計画を出させ、レビューしてから通常モードで実装
- 研究コードでは特に有効：解析手順の妥当性を先に確認できる
- 表示：画面下に「 **plan mode on**」と出る

その他の操作：Esc で生成を中断 / Ctrl+C を 2 回で終了 / 「!」 始まりでシェルコマンドを直接実行 / 「@ファイル名」で特定ファイルを参照させる / 画像はドラッグ&ドロップで貼れる。

入力欄で「/」を打つと一覧が出る。まずはこの 9 つで十分

コマンド	役割	いつ使う
/init	CLAUDE.md を自動生成	新しいプロジェクトで最初に 1 回
/help	コマンド一覧・ヘルプ	迷ったとき
/clear	会話履歴をリセット	別のタスクに移るとき（コンテキストの混線を防ぐ）
/compact	会話を要約して圧縮	長い作業の途中で応答が鈍ってきたとき
/context	コンテキスト使用量を表示	どれだけ消費しているか確認したいとき
/model	使用モデルの切替	精度重視（Opus）か速度・コスト重視（Sonnet）か
/memory	CLAUDE.md 等のメモリを編集	ルールを追記したいとき
/agents	Subagent の一覧・管理	§3 後半で使用
/doctor	インストール・設定の診断	動作がおかしいとき

Skills や Subagent も「/名前」で呼び出せる（後述）。よく使う自作の手順はスラッシュコマンド化しておくチームで共有しやすい。

会話が長くなると精度が落ち、コストも増える。「見える化」して「圧縮」するのが基本動作

/context 何がコンテキストを占めているかを見る

```
> /context
System prompt      | 3.2k
CLAUDE.md + Skills | 4.8k
Messages           | 61.5k ← 大半はファイル読込
Total 69.5k / 200k (35%)
```

こう読む

- Messages が肥大 → 読んだファイルやログが溜まっている
- CLAUDE.md + Skills が大きい → ルールの棚卸しを
- 70～80% を超えたら /compact のタイミング

/compact 会話を要約して続ける

```
> /compact 実装済みの内容と未解決のテスト失敗を残して
Compacting conversation...
✓ 69.5k → 8.1k tokens

> /clear ← 別タスクに移るときは要約せず全消去
```

使い分け

- /compact：同じタスクの続き。引数で「残すもの」を指示
- /clear：別タスクへ。決定事項は先に CLAUDE.md へ書く
- 自動圧縮もあるが、区切りで自分から打つ方が要約が良い

目安：1 タスク = 1 セッション。「読ませる → 計画 → 実装 → テスト」が終わったら /compact か /clear。Subagent を使うと調査分をメインに載せずに済む (§ 3)。

「目的・入出力・制約・完了条件」を書く。人に頼むときと同じ

× 悪い例

「欠損値の処理を追加して」

何が起きるか

- どの関数？ どの列？ 方法は？ → Claude が推測で決める
- テストがないので「動いた」の基準がない
- 後から直す手戻りが増える

研究では特に危険：「もっともらしく動く誤った解析」が最悪の結果

load_data.py の load_csv() に欠損値処理を追加したい。

- 数値列は列ごとの中央値で補完、文字列の列は "unknown" で埋める
- 引数 `fill_missing: bool = True` で ON/OFF できるように
- 既存の呼び出し側はコード修正なしで動くこと (デフォルト True)
- tests/test_load.py に次を検証するテストを追加
 1. デフォルトで欠損が0個になり、数値列は中央値、文字列の列は"unknown"で埋まっていることを確認
 2. `fill_missing=False`なら欠損がそのまま残る
- pytest が全部通ることが完了条件

まずコードを読んで変更計画だけ提示して。実装はその後で。

○ 良い指示の 4 要素

目的（何のため）／入出力（どの関数・列・型）／制約（変えてはいけないもの）／完了条件（テストが通る・図が出る）＋「まず計画を出して」

コツ：うまくいかなかったら指示を足すのではなく、CLAUDE.md にルールとして書く (§3)。同じ注意を 2 回したら CLAUDE.md 行き。

ゴール：load_csv() に欠損値処理を追加し、pytest を全部通す（読む→計画→実装→検証 の分割を体験）

1

プランモードに切替（Shift+Tab）、前スライドの指示を貼る

→ Claude がコードを読んで計画を提示する。書き換えは起きない

2

計画をレビューして修正指示

例：「中央値は train 側だけで計算して」「型ヒントを付けて」

3

通常モードに戻して「その計画で実装して」

→ 差分が順に提示される。1 つずつ内容を読んで承認

4

Claude が pytest を実行 → 失敗があれば自己修正

→ 赤→緑になる過程を観察。勝手に test を消していないか確認

5

最終確認とコミット

git diff で全体を眺める → commit & push（ここは人力にするべき）

学習ポイント：完了条件を「テストが通る」にすると自走しやすい／承認前に必ず差分を読む／Claude が書いたテストは「何を検証しているか」を自分で説明できること。

Antigravity の場合：同じ指示文をそのまま使える。「計画だけ提示」は Planning モードを選ぶ。差分はアプリ内ビューで確認して承認する。

毎セッションの最初に自動で読み込まれる。ここに書いたルールは、毎回言わなくても守られる

書くべきこと

- 環境：ライブラリや仮想環境の管理（venvなど）
- 実行コマンド：テスト・lint・解析の起動方法
- ディレクトリ構成：data/ は読み取り専用、results/ へ出力
- コーディング規約：型ヒント必須、ruff 準拠、日本語 docstring
- 解析のルール：乱数 seed 固定、生データを上書きしない
- 「2 回注意したこと」を全部ここに追記

運用

- 100～300 行が目安。長すぎると読まれなくなる
- Git にコミットして研究室で共有／古い記述は消す

```
# CLAUDE.md
## 環境
- Python >=3.9。依存は pyproject.toml (pip install -e .)
- 実行: pytest / python stats.py

## ディレクトリ
- data/raw は絶対に変更しない（読み取り専用）
- 図は results/figures/ に PDF と PNG の両方で保存

## コーディング規約
- すべての関数に型ヒントと日本語 docstring
- 乱数は numpy の default_rng(seed=0) を使う
- pandas の chained assignment は禁止

## 作業の進め方
- 変更前に必ず計画を提示する
- テストを追加せずに機能追加しない
- Git操作 (commitやpush) は禁止
```

階層： ~/.claude/CLAUDE.md（個人・全プロジェクト共通） → プロジェクト直下の CLAUDE.md（チーム共有） → サブディレクトリの CLAUDE.md（部分ルール）。上書きではなく合成される。

CLAUDE.md は「常を守るルール」、Skill は「必要なときに呼び出す作業手順書」

仕組み

- フォルダ 1 つ + SKILL.md 1 枚 (Markdown) で定義
- 呼び出し：「/skill名」で明示、または依頼内容に合えば Claude が自動で使う
- 普段はメタ情報だけ読み、使うときに本文を読む → コンテキストを節約

2 つの置き場所

~/.claude/skills/<name>/SKILL.md

個人用。全プロジェクトで使える

.claude/skills/<name>/SKILL.md

プロジェクト用。Git でコミットすれば研究室全員に配布される ← 本日のおすすめ

CLAUDE.md / Skill / 毎回の指示 の使い分け

CLAUDE.md

プロジェクト全体に常に効くルール。「data/raw を変更しない」「型ヒント必須」

Skill

特定の作業の手順。「論文用の図を描く」「解析レポートを作る」「PDF を要約する」

その場の指示

今回だけの内容。「この CSV の 3 列目を対数変換して」

判断基準：3 回以上繰り返す手順 → Skill 化。2 回注意したルール → CLAUDE.md

Skills は Anthropic が公開した「Agent Skills」というオープン規格。同じ SKILL.md が Claude.ai のチャットや他社の AI コーディングツールでも動く。

上の YAML が「いつ使うか」、下の Markdown が「何をするか」。description の書き方で Claude の参照頻度が変わる

```
---
name: plot-style
description: 研究室の論文投稿規約で matplotlib の図を作成・修正する。
            ユーザが「論文用の図」「figure を整えて」「投稿規定に合わせて」と言ったら使う。
---

# 論文用プロット規約

## 手順
1. `references/style.mplstyle` を読み、plt.style.use で適用する
2. フォントは Arial 8pt、線幅 1.0、カラーは colorblind-safe パレット
3. 図サイズは 1 段 85 mm / 2 段 175 mm 幅で作成
4. results/figures/ に PDF と PNG (300 dpi) を保存
5. 保存後、生成した図を開いて軸ラベル・単位・凡例を確認する

## 禁止事項
- タイトルを図内に書かない（キャプションで書く）
- デフォルトの tab10 カラーを使わない
```

name

呼び出し名。/plot-style で明示実行できる。小文字とハイフン

description = 使用条件

ユーザが実行しなくても、Claude が「使うべきか」を判断する条件。何をするか + ユーザがどう言ったときか、を具体的に書く。曖昧だと自動で使用されない

本文 = 手順書

番号付き手順 + 禁止事項。長い参考資料は references/、決定的処理は scripts/ に分けて「読め」と指示する

構成例：.claude/skills/plot-style/{SKILL.md, references/style.mplstyle, scripts/check_figure.py}。まず SKILL.md 1 枚から始め、必要になったら足す。

「毎回同じ手順・同じ品質」が求められる作業ほど Skill 向き。研究室で共有すれば新人の立ち上げも速くなる

/plot-style

論文用の図を規約通りに描く

フォント・サイズ・カラー・保存形式を統一。投稿先ごとに references/ を切り替え

「Fig.2 を Nature 形式で描き直して」

/analysis-report

解析 → Markdown レポート

データ概要・前処理・統計・図を定型フォーマットで出力。ゼミ資料の下書きに

「今日の結果を /analysis-report にまとめて」

/paper-summary

PDF 論文の構造化要約

目的・手法・結果・限界・自分たちの研究との関係、を固定項目で抽出

「この PDF を /paper-summary で」

/new-experiment

実験フォルダの雛形生成

config.yaml・README・解析ノートブック・tests を規約通りに作成

「exp_2026_09_visual を作って」

/commit

規約に沿ったコミット

Conventional Commits 形式、日本語本文、関連 issue の紐付け

「/commit」

/review-analysis

解析コードの妥当性チェック

seed 固定・リーク・多重比較・単位の混在を観点リストで点検

「stats.py を /review-analysis して」

運用の提案：研究室リポジトリに .claude/skills/ を置き、PR で追加・改訂する。Skill が「効かない」ときは description を具体化する。

ゴール：ラボ規約で図を描く Skill を作り、自動発火と明示呼び出しの両方を確認する

1	Skill を Claude 自身に書かせる	「.claude/skills/plot-style/SKILL.md を作って。内容は...（前々スライドの要件を貼る）」
2	生成された SKILL.md を確認・修正	description に「論文用の図」「投稿規定」などの言い回しが入っているか
3	明示呼び出し or 自動使用	/plot-style で results/summary.csv の群間比較を棒グラフにして or （Skill 名を言わずに）論文用にresults/summary.csvの群間比較を棒グラフで作って
4	注意！	上記のSkillはclaude起動時に自動で読み込まれるので，作成したばかりのSkillは参照されない。試して見るときは，/clearで会話をリセットor新しいclaudeを立ち上げてみる
5	コミットして共有	git add .claude/skills → git commit → 研究室の全員が同じ Skill を使える

うまく発火しないとき：description が抽象的／Skill フォルダが新規作成直後（セッション再起動で認識）／同名の Skill が個人用と衝突。/help で Skill 一覧を確認できる。

Antigravity の場合：置き場所を .agents/skills/plot-style/SKILL.md に読み替える。SKILL.md の中身（frontmatter+手順）は完全に同じ。

Subagents とは：専門担当を切り出して「別のコンテキスト」で働かせる

19

メインの Claude が「レビュー担当」「テスト担当」など専任のサブエージェントに仕事を委任し、結果だけを受け取る

メイン セッション
あなたと会話・全体を統括

↓ 委任 ↑ 結果

reviewer
読む・指摘のみ
Read, Grep

tester
テスト作成
Read, Edit, Bash

explorer
調査専門
内蔵

なぜ分けるか

- コンテキスト独立：メインが汚れない
- ツール制限：レビュー担当は編集不可
- 役割ごとに専用の指示を固定できる
- 並列実行：調査と実装を同時進行

```
# .claude/agents/code-reviewer.md
---
name: code-reviewer
description: 解析コードの妥当性レビュー担当。統計手法・
データリーク・再現性の観点で指摘する。コードの変更は行わない。
実装が終わった直後や「レビューして」と言われたら必ず使う。
tools: Read, Grep, Glob
model: sonnet
---
```

あなたは研究室のシニア研究員として解析コードをレビューする。

観点

- 乱数 seed が固定されているか
- train/test のリーク（前処理の統計量を全データで計算していないか）
- 多重比較の補正、単位・スケールの混在
- data/raw への書き込みがないか

出力形式

重大 / 中 / 軽微 に分類し、ファイル名と行番号を添えて箇条書き

Skill との違い：Skill は「手順書」をメインが読んで実行する。Subagent は「別人格・別コンテキスト」で実行し結果だけ返す。置き場所は .claude/agents/（プロジェクト）または ~/.claude/agents/（個人）。

ハンズオン ④：Subagent に「レビュー」と「テスト」を分担させる

20

ゴール：編集できないレビュー担当と、テスト専任の担当を定義し、ハンズオン②のコードを 2 人体制で仕上げる

1	2 つの Subagent を Claude に書かせる	「.claude/agents/ に code-reviewer を（前スライドの内容）で作って」
2	レビューを委任する	「code-reviewer に load_data.py と stats.py をレビューさせて。結果を重大度順にまとめて」
3	/list-agents で一覧を確認	Subagents が認識されているか確認可
ついでに		/context でメインの消費量を確認。大量の読み込みがメインに入っていないことを見る

設計の原則：レビュー担当には Edit/Write を渡さない（最小権限）／description に「いつ使うか」を書けば、次回からは「レビューして」だけで自動委任される。

Antigravity の場合：定義ファイルの形式が異なるため、本日はデモを観察して「役割分担・最小権限・コンテキスト分離」の考え方を持ち帰る。

今日は「こういうものがある」と知っておけば十分。必要になったときに公式ドキュメントへ

Hooks

決まったタイミングで自動実行するコマンド

研究室での用途

編集のたびに ruff format を自動実行／
危険なコマンドをブロック／作業終了時
に通知

```
# .claude/settings.json
"hooks": {
  "PostToolUse": [{
    "matcher": "Edit|Write",
    "hooks": [{ "type": "command",
      "command": "ruff format $FILE"
    }]
  }]
}
```

ヘッドレス実行

対話なしで 1 回実行し、結果を出力

研究室での用途

毎朝のデータ品質チェック／CI でのコー
ドレビュー／論文要約のバッチ処理

```
# 1 回だけ質問して終了
claude -p "テスト失敗の原因を要約"

# パイプで渡す
cat results.log | claude -p ¥
"異常値をリストにして"

# cron / GitHub Actions から呼べる
```

MCP (Model Context Protocol)

外部ツールを接続するための共通規格

研究室での用途

ブラウザ操作 (Playwright) ／GitHub
の issue 操作／DB 照会／論文 DB 検索

```
# サーバーを登録
claude mcp add playwright -- ¥
npx @playwright/mcp@latest

# 以降、会話の中で使える
「論文 DB を開いて検索し、
上位 5 件を表にまとめて」
```

優先順位の目安：CLAUDE.md → Skills → 権限設定 → Subagents → Hooks → MCP。左から順に整えると、少ない手間で見え方が大きい。

自律的に動く = ファイルとシェルに直接触れる。便利さと同じだけの注意が要る

1	権限を設定してから使う	allow : pytest · ruff · git status / ask : git commit · pip install / deny : rm -rf · git push --force · data/raw への書き込み
2	差分は必ず読んで承認する	「全部 Yes」は禁止。特に解析ロジック（統計・前処理）の変更は自分で説明できるまで承認しない
3	秘匿データを渡さない	被験者データ・個人情報・API キー・未発表結果は .gitignore と deny で守る。権限スキップ起動オプションは使わない
4	Git の上で作業する	作業前にブランチを切る。何かあれば git checkout で戻せる状態を保つ。コミットは小さく
5	理解したコードだけ使う	「動いた」 ≠ 「正しい」。読んで説明できるコードだけを研究成果に使う。学生は指導教員に説明できること

```
# .claude/settings.json
{
  "permissions": {
    "allow": [
      "Bash(pytest*)",
      "Bash(ruff*)",
      "Bash(git status*)",
      "Bash(git diff*)"
    ],
    "ask": [
      "Bash(git commit*)",
      "Bash(pip install*)",
      "Bash(uv add*)",
      "Bash(conda install*)"
    ],
    "deny": [
      "Bash(rm -rf*)",
      "Bash(git push --force*)",
      "Write(data/raw/**)",
      "Read(.env)"
    ]
  }
}
```

コスト：Pro は利用上限が早く来る。Subagent や並列実行は消費が大きい。/context と /compact でコンテキストを管理し、Skill を使うと繰り返しの説明コストが下がる。

道具は「使い方を共有する仕組み」があって初めて研究室の資産になる

1

タスク単位で任せ、承認で制御する

読む → 計画 → 実装 → 検証。プランモードと差分レビューが品質の要

2

知識はファイルに書いて共有する

CLAUDE.md (ルール) ・ Skills (手順) ・ Subagents (役割) を Git で研究室に配布

3

安全は設定で担保する

権限の allow / ask / deny、データ保護、理解できないコードは使わない

研究室での運用（提案）

- 共通リポジトリで .claude/ 一式を管理・配布
- Skill・Subagent の追加は PR で見える化
- 月 1 回、CLAUDE.md と Skills を棚卸し
- 学生は生成コードをゼミで自分の言葉で説明

次回までの課題

- 自分の研究プロジェクトで /init → CLAUDE.md を研究室版に書き直す
- 3 回以上繰り返している作業を 1 つ Skill 化して PR を出す
- 権限設定 (settings.json) を自分の環境に合わせて作る

参考：code.claude.com/docs (公式ドキュメント) / github.com/anthropics/skills (公開 Skills 集)

質問・相談はいつでも。うまくいった Skill やプロンプトは研究室 Slack の #claude-code チャンネルで共有してください。